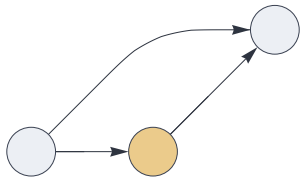
Getting Started

This document serves as a practical introduction to defining neural circuits within the *Creating Intelligence* framework. Circuits are compiled into stateful machines that process hyperdimensional sparse distributed representations (SDRs) and sparse holographic representations (SHRs).

A JSON dataflow description language fully specifies circuits composed of standardized building blocks:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [11]},
    {"component": "output", "receive": [[11, 21]]}
  ]}
```



Built-in functionality can be extended via plug-in mechanisms for user-defined circuit components and signal encoding/decoding functions.

All examples shown here have been generated programmatically. This ensures that the dataflow specifications and test inputs are error-free, and that the test outputs are an accurate representation of the package's functionality.

Build date: Friday 11 September 2026 14:14:21

Input and Output

Here is the most basic circuit, routing data directly from the input to the output:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [11]}
  ]}
```

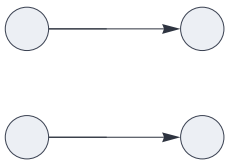
The source of each pathway is a numbered slot. Slot numbers can be choosen arbitrarily. Hyperparameters are assigned to slots. In the above example, a dimension N = 1000 and a set population P = 10 are specified as the default setting for all slots.

This setup simply echoes its input:

```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

Circuits may contain multiple inputs and multiple outputs, with different hyperparameters:

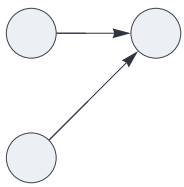
```
{ "hyperparameters": {"default": [1000, 10], "12": [900, 9]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "input", "send": [12]},
    {"component": "output", "receive": [11]},
    {"component": "output", "receive": [12]}
  ]}
```



```
> {1, 2, 3, 4, 5}, {6, 7, 8, 9, 10}
{1, 2, 3, 4, 5}, {6, 7, 8, 9, 10}
```

Here, two inputs are routed to the same output.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "input", "send": [12]},
    {"component": "output", "receive": [[11, 12]]}
  ]}
```

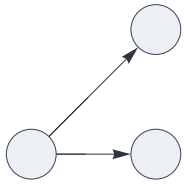


The two pathways are merged:

```
> {1, 2, 3, 4, 5}, {6, 7, 8, 9, 10}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

This broadcasts one input to two outputs:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [11]},
    {"component": "output", "receive": [11]}
  ]}
```



The output is a sequence of two representations:

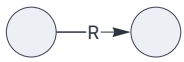
```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}, {1, 2, 3, 4, 5}
```

Circuit Connections

The data paths connecting circuit components can be flexibly configured to transport, transform, route and merge data. In the dataflow description, pathway options are given as tags wtihin the the components' "receive" property.

In the following example, the pathway is tagged with "rate_limit":

```
{ "hyperparameters": {"11": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [{"slot": 11, "tags": ["rate_limit"]}]}
  ]}
```



The input is subsampled to the population specified in the hyperparameter setting.

```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
{4, 5, 7, 8, 9}
```

Pathways convey either sets (the default) or multisets. Multiset pathways may carry duplicate elements, as well as negative entries indicating inhibitory signals.

Multiset paths are displayed as thick colored lines:

```
{ "hyperparameters": {"11": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [{"slot": 11, "tags": ["multiset"]}]}
  ]}
```

```
> {1, 1, 2, 3, 4, 4}
{1, 1, 2, 3, 4, 4}
```

Reference: Dataflow Options

Circuit visualization

⌗	"show_slot"	displays the path's slot number
$	"show_dimension"	displays the path's dimension hyperparameter (N)
%	"show_population"	displays the path's population hyperparameter (P)

Runtime logging

?	"log"	prints the path's payload to the console in each evaluation cycle

Path modifiers

+	"multiset"	allows duplicates and negative elements
−	"inhibit"	multiset path with positive elements flipped to negative

Temporal gating

@	temporal gate	opens or closes the path with respect to the circuits cycle count

Data transformation

π	"permute"	applies a permutation unique to the path
R	"rate_limit"	limits the population to the path's hyperparameter
T	"threshold"	clears data below hyperparameter population
~	"noise"	generates random noise if signal is non−empty, else clears data

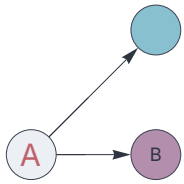
Data routing and merging

X	"veto"	clears all paths if any X path is non−empty
*	"mandatory"	clears all paths if any * path is empty
!	"priority"	clears all non−! paths if any ! path is non−empty
&	"dependency"	clears & paths if any non−& path is empty (AND)
|	"fallback"	clears | paths if any non−| path is non−empty (NOR)
=	"barrier"	clears = paths if any = path is empty
K	"kwta_excitatory"	top−k excitatory (positive) signals
k	"kwta_absolute"	top−k signals (total saliency)

Circuit Visualization

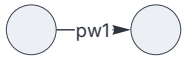
We can customize the rendering style of circuit components with display labels, different font sizes and colors, and background fill colors. The system uses a standard color palette with colors numbered from 0 to 15.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11], "label": "A", "color": 11, "size": 20},
    {"component": "output", "receive": [11], "label": "B", "fill": 15, "size": 12},
    {"component": "output", "receive": [11], "fill": 8}
  ]}
```



This shows a custom pathway label:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [{"slot": 11, "label": "pw1"}]}
  ]}
```



Reveal the path's slot number:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [{"slot": 11, "tags": ["show_slot"]}]}
  ]}
```

Display the path's data dimension:

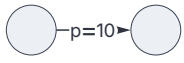
```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [{"slot": 11, "tags": ["show_dimension"]}]}
  ]}
```

Display the paths's population:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [{"slot": 11, "tags": ["show_population"]}]}
  ]}
```

Combine a custom label with the population:

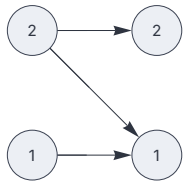
```
{ "hyperparameters": {"default": [900, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive":
   [{"slot": 11, "label": "p=", "tags": ["show_population"]}]}
  ]}
```



Multiple input and output components realize block-coded inputs and outputs. The block position is determined by the order of

appearance of input and output components. This reveals their respective sequence positions:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11], "label": "#"},{"component":
  "input", "send": [12], "label": "#"},{"component": "output", "receive": [[11,
  12]], "label": "#"},{"component": "output", "receive": [12], "label": "#"}  ]}
```



Tagging a connection with “log” prints its value at each cycle to the console:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [1]},
    {"component": "output", "receive": [{"slot": 1, "tags": ["log"]}]}
  ]}
```


```
Slot 1: {1, 2, 3, 4, 5}
```

```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

Symbolic Token Encoding

Input nodes can specify an encoder function that converts external inputs to sparse sets. Similarly, output nodes can decode internal representations. This software framework includes a library of commonly used encoders and decoders. Additional, user-defined encoders can be plugged in directly without requiring modifications of the core framework. This mechanism can also be used for user-defined preprocessing and postprocessing functionality and connections to external data sources.

Generic token encoder and decoder

The “codec” module encodes distinct input data into random SHRs. This is the standard method of representation symbolic tokens. Plugged into an output node, it decodes the set to the original data format.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [11]},
    {"component": "output", "plugin": "codec", "receive": [11]}
  ]}
```

Encode and decode a string:

```
> Hello, World!
Hello, World!
```

Dropping the decoding step exposes the underlying SHR:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [11]},
    {"component": "output", "receive": [11]}
  ]}
```

Encode a string:

```
> apple
{167, 176, 227, 350, 464, 574, 611, 671, 697, 847}
```

The Null token is encoded as an empty list:

```
> Null
{}
```

Category encoding

For encoding a small number of categories, non-overlapping representations often works better than the generic symbolic encoder discussed above. Here we choose a population P=10 to encode K=10 categories numbered 0 to 9. This requires a minimum dimension N = 100:

```
{ "hyperparameters": {"default": [100, 10]},
  "dataflow": [
    {"component": "input", "plugin": "category", "send": [11], "categories": 10},
    {"component": "output", "receive": [11]}
  ]}
```


```
> 3
{31, 32, 33, 34, 35, 36, 37, 38, 39, 40}
```

If the specified dimension N is greater than K*P, the category encoder generates a random sample of the available elements:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "plugin": "category", "send": [11], "categories": 10},
    {"component": "output", "receive": [11]}
  ]}
```

A random encoding, sampled from 100 possible elements:

```
> 3
{316, 317, 319, 328, 330, 363, 365, 389, 390, 398}
```

A subsequent evaluation generates a different encoding:

```
> 3
{304, 310, 312, 313, 316, 317, 338, 355, 363, 388}
```

Use the category encoder/decoder in the input and the output node:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "plugin": "category", "send": [11], "categories": 10},
    {"component": "output", "plugin": "category",
   "receive": [{"slot": 11, "tags": ["log"]}], "categories": 10}
  ]}
```

Tagging the connection with “log” reveals the internal representation:

```
Slot 11: {717, 723, 725, 733, 736, 750, 757, 759, 785, 791}
```

```
> 7
7
```

The “binning” decoder maps similar representations to bins, enumerated by integers 1,2,...
Similarity is defined via the pattern matching threshold implied by the data’s hyperparameters.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "plugin": "binning", "receive": [11]}
  ]}
```

This assigns bin number 1 to an SDR:

```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
1
```

A dissimilar SDR:

```
> {11, 12, 13, 14, 15, 16, 17, 18, 19, 20}
2
```

This input falls into the first cluster:

```
> {1, 2, 3, 4, 5, 6, 7, 88, 99, 100}
1
```

Encoding Numeric Data

This framework includes multiple SDR encoders for numeric data.

Vector encoding

The general-purpose vector encoder transforms dense numeric data to SDRs. It performs a random projection onto a hyperdimensional space by multiplying the input with a random sparse binary matrix, followed by kWTA to control the output sparsity. This implementation of kWTA, using the connection's population as the value of k, may return more than k elements in case of a tie between the top-k elements.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "plugin": "vectorencoder", "send": [11]},
    {"component": "output", "receive": [11]}
  ]}
```

Encode a dense vector:

```
> {0.339, 0.922, 0.164, 0.998, 0.012, 0.972, 0.717, 0.305}
{98, 184, 269, 413, 502, 509, 553, 630, 713, 988}
```

A similar vector maps to the same sparse output:

```
> {0.329, 0.923, 0.181, 0.98, 0.031, 0.954, 0.708, 0.318}
{98, 184, 269, 413, 502, 509, 553, 630, 713, 988}
```

An empty list remains unencoded:

```
> {}
{}
```

Flyhash encoding

A variant of the general-purpose vector encoding discussed above, the “flyhash” algorithm encodes a dense vector into a sparse binary representation.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "plugin": "flyhash", "send": [11]},
    {"component": "output", "receive": [11]}
  ]}
```

Encode a dense vector:

```
> {0.74, 0.52, 0.04, 0.65, 0.81, 0.73, 0.66, 0.19, 1., 0.81, 0.87, 0.2}
{73, 100, 143, 190, 193, 289, 416, 722, 805, 890}
```

A similar vector maps to the same SDR:

```
> {0.68, 0.47, 0.04, 0.6, 0.79, 0.67, 0.63, 0.25, 1.08, 0.78, 0.79, 0.26}
{73, 100, 143, 190, 193, 289, 416, 722, 805, 890}
```

An entirely different vector maps to an orthogonal SDR:

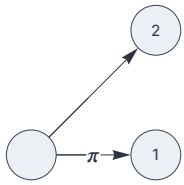
```
> {0.45, 0.27, 0.82, 0.64, 0.36, 0.02, 0.51, 0.2, 0.27, 0.98, 0.9, 0.57}
{74, 102, 106, 206, 215, 405, 419, 778, 846, 910}
```

Permutations

Pathways naturally don’t preserve the order of elements in a signal. This is modeled by applying set permutations. We can configure specific pathways to perform distinct random permutations.
In this framework, permutations are irreversible.

Apply a permutation to the first output:

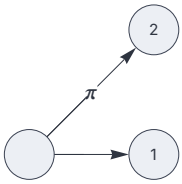
```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [{"slot": 11, "tags": ["permute"]}],
  "label": "#"},{"component": "output", "receive": [11], "label": "#"}  ]}
```




```
> {1, 2, 3, 4, 5}
{288, 302, 350, 512, 942}, {1, 2, 3, 4, 5}
```

Apply a permutation to the second output. The order in which input and output nodes appear in the dataflow description matches the order of the respective input and output sequences.

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [11], "label": "#"},{"component":
  "output", "receive": [{"slot": 11, "tags": ["permute"]}], "label": "#"}  ]}
```




```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}, {246, 329, 337, 356, 698}
```

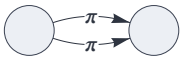
Bundling a set with its own permutation:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [[11, {"slot": 11, "tags": ["permute"]}]]}
  ]}
```


```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5, 72, 278, 358, 665, 981}
```

Different pathways apply different, unique permutations:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [[{"slot":
  11, "tags": ["permute"]}, {"slot": 11, "tags": ["permute"]}]]}
  ]}
```



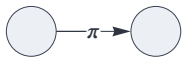

```
> {1, 2, 3, 4, 5}
{13, 84, 129, 147, 241, 523, 667, 706, 968, 999}
```

Nested Circuits

This framework allows circuits to be logically nested, enabling modular designs based on reusable subnets.

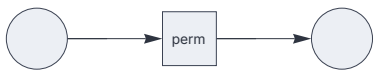
Here we create a circuit that simply permutes its input:

```
{ "options": {"name": "perm"},
  "hyperparameters": {"default": [200, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [{"slot": 11, "tags": ["permute"]}]}
  ]}
```



This circuit embeds the subnet, referenced by the “name” property:

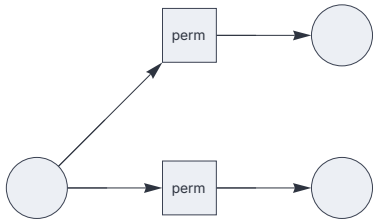
```
{ "hyperparameters": {"default": [200, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "circuit", "send": [21], "receive": [11], "name": "perm"},
    {"component": "output", "receive": [21]}
  ]}
```




```
> {1, 2, 3, 4, 5}
{36, 58, 174, 187, 190}
```

The same subnet instance can be shared by multiple circuit nodes:

```
{ "hyperparameters": {"default": [200, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "circuit", "send": [21], "receive": [11], "name": "perm"},
    {"component": "circuit", "send": [22], "receive": [11], "name": "perm"},
    {"component": "output", "receive": [21]},
    {"component": "output", "receive": [22]}
  ]}
```

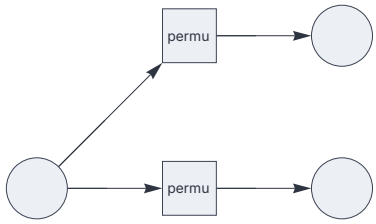


Note that this applies the exact same permutation in both network branches:

```
> {1, 2, 3, 4, 5}
{36, 58, 174, 187, 190}, {36, 58, 174, 187, 190}
```

Embedded circuit specifications can be imported from a JSON file. The following creates two distinct instances based on the same specification. Hyperparameters of the embedded circuit are automatically rescaled to match the embedding circuit.

```
{ "hyperparameters": {"default": [200, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "circuit", "plugin":
  "file", "send": [21], "receive": [11], "name": "permutation"},
    {"component": "circuit", "plugin":
  "file", "send": [22], "receive": [11], "name": "permutation"},
    {"component": "output", "receive": [21]},
    {"component": "output", "receive": [22]}
  ]}
```



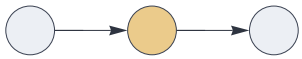
The two branches apply different permutations:

```
> {1, 2, 3, 4, 5}
{11, 43, 45, 167, 187}, {15, 36, 64, 155, 156}
```

Delay

Insert a delay component into the dataflow:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [11]},
    {"component": "output", "receive": [21]}
  ]}
```



This echoes the input immediately, because connections from the input and to the output transport signals without delay:

```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

A series of two delay components:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [11]},
    {"component": "delay", "send": [31], "receive": [21]},
    {"component": "output", "receive": [31]}
  ]}
```

This delays the dataflow by one cycle, as it takes one step for the signal to travel from the first delay component to the second:

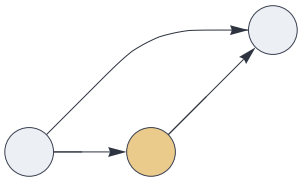
```
> {1, 2, 3, 4, 5}
{}
```

```
> {6, 7, 8, 9, 10}
{1, 2, 3, 4, 5}
```

```
> {}
{6, 7, 8, 9, 10}
```

In this setup, both paths transmit the signal simultaneously, because input and output connections are instantaneous.

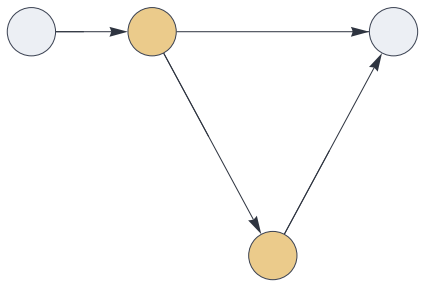
```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [11]},
    {"component": "output", "receive": [[11, 21]]}
  ]}
```



```
> {1, 2, 3, 4, 5}
{1, 1, 2, 2, 3, 3, 4, 4, 5, 5}
```

This bundles two consecutive inputs:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [11]},
    {"component": "delay", "send": [31], "receive": [21]},
    {"component": "output", "receive": [[21, 31]]}
  ]}
```



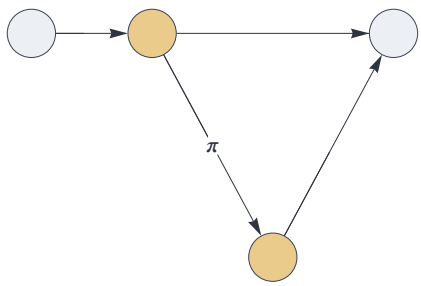
```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

```
> {6, 7, 8, 9, 10}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

```
> {}
{6, 7, 8, 9, 10}
```

In the above example, the order of consecutive inputs is not preserved in the output. Inserting a permutation encodes the order:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [11]},
    {"component": "delay", "send": [31], "receive": [{"slot": 21, "tags": ["permute"]}]},
    {"component": "output", "receive": [[21, 31]]}
  ]}
```



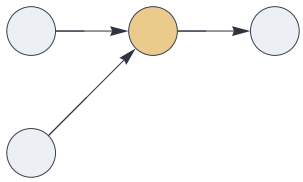

```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

The delayed input is permuted:

```
> {6, 7, 8, 9, 10}
{6, 7, 8, 9, 10, 18, 277, 463, 693, 772}
```

Delay components may receive multiple blocks and send multiple blocks. This example produces the disjoint bundle of two sets:

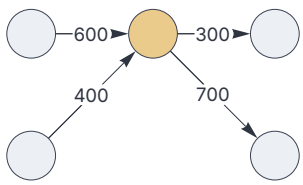
```
{ "hyperparameters": {"default": [1000, 5], "101": [2000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "input", "send": [12]},
    {"component": "delay", "send": [101], "receive": [11, 12]},
    {"component": "output", "receive": [101]}
  ]}
```



```
> {1, 2, 3, 4, 5}, {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5, 1001, 1002, 1003, 1004, 1005}
```

The total dimension of the input blocks must match the total dimension of outputs:

```
{ "hyperparameters": {"11": [400, 10], "12": [600, 10], "101": [700, 10], "102": [300, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "input", "send": [12]},
    {"component": "delay", "send": [101, 102], "receive": [{"slot":
  11, "tags": ["show_dimension"]}, {"slot": 12, "tags": ["show_dimension"]}]},
    {"component": "output", "receive": [{"slot": 101, "tags": ["show_dimension"]}]},
    {"component": "output", "receive": [{"slot": 102, "tags": ["show_dimension"]}]}
  ]}
```




```
> {1, 2, 3, 4, 5}, {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5, 401, 402, 403, 404, 405}, {}
```

Temporal Integration

The “delay” circuit component can accumulate signals up to a configurable temporal capacity. By default, it updates the internal state with the incoming signal through a multiset union operation (the “augmentation” update rule).

In this example, the internal capacity is twice the default population:

```
{ "hyperparameters": {"11": [1000, 5], "21": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [11], "capacity": 2.0},
    {"component": "output", "receive": [21]}
  ]}
```

This echoes the input immediately:

```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

Carry over the previous input:

```
> {6, 7, 8, 9, 10}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

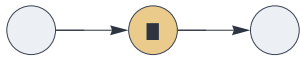
With additional input, capacity-bound subsampling is applied:

```
> {11, 12, 13, 14, 15}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15}
```

Sample-and-Hold

The “latch” component emits the last non-empty signal when it receives an empty input.

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "latch", "send": [21], "receive": [11]},
    {"component": "output", "receive": [21]}
  ]}
```



```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

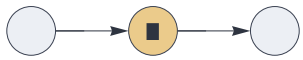
Empty input triggers a repeated emission of the last salient signal:

```
> {}
{1, 2, 3, 4, 5}
```

```
> {}
{1, 2, 3, 4, 5}
```

The latch component with a specified time window and signal threshold:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "latch", "send": [21], "receive": [11], "cycles": 2, "threshold": 0.8},
    {"component": "output", "receive": [21]}
  ]}
```



```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

Sub-threshold input:

```
> {10, 11, 12}
{1, 2, 3, 4, 5}
```

Now the time window has expired:

```
> {20, 21, 22}
{}
```

Multiplexing and Temporal Gating

By default, invoking a circuit's function triggers one evaluation cycle. Multiple cycles with the same input, or alternatively with zero inputs (an empty list for each function argument) can be triggered by setting the "multiplex" option in the circuit's dataflow description. The multiplexing specification is a list of integers 0 or 1. Each 1 triggers an evaluation cycle with the same repeated input. Each 0 triggers a cycle with empty inputs, during which only the circuit-internal state is processed.

This circuit triggers three cycles with the same input. We tag the path with "log" to inspect the dataflow:

```
{ "options": {"multiplex": [1, 1, 1]},
  "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [{"slot": 11, "tags": ["log"]}]}
  ]}
```

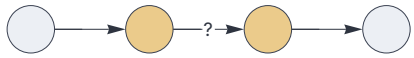

```
Slot 11: {1, 2, 3, 4, 5}
Slot 11: {1, 2, 3, 4, 5}
Slot 11: {1, 2, 3, 4, 5}
```

```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

Zero-padding, specified by a list [1, 0, ...] is typically used to flush the delay line in embedded circuits that take multiple cycles to process an individual input.

With the following dataflow, inputs by default are delayed by one cycle. Zero-padding with one additional cycle removes the delay:

```
{ "options": {"multiplex": [1, 0]},
  "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [11]},
    {"component": "delay", "send": [31], "receive": [{"slot": 21, "tags": ["log"]}]},
    {"component": "output", "receive": [31]}
  ]}
```




```
Slot 21: {}

Slot 21: {1, 2, 3, 4, 5}
```

```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

Paths tagged with a “gating” option act as temporal gates with respect to the circuit’s cycle counter. This circuit lets its input pass through in two out of three cycles:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [{"slot": 11, "gating": [1, 1, 0]}]}
  ]}
```

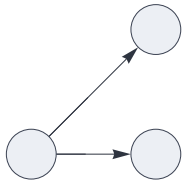
```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

```
> {1, 2, 3, 4, 5}
{}
```

This circuit multiplexes inputs between the two branches:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [{"slot": 11, "gating": [1, 0]}]},
    {"component": "output", "receive": [{"slot": 11, "gating": [0, 1]}]}
  ]}
```



```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}, {}
```

```
> {1, 2, 3, 4, 5}
{}, {1, 2, 3, 4, 5}
```

Inhibition

Positive set elements (i > 0) denote excitatory neurons, whereas their negative counterparts (-i) are inhibitory. All circuit components properly handle the presence of negative elements. Only multiset paths (tagged "multiset") convey inhibitory signals.

In this framework, the only source of negative set elements (apart from external inputs) are paths tagged as "inhibit". This circuit flips positives into negatives, preserving the negatives:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [{"slot": 11, "tags": ["inhibit"]}]}
  ]}
```

Note that the result is a multiset.

```
> {1, 2, 3, -3, -4, -5}
{-5, -4, -3, -3, -2, -1}
```

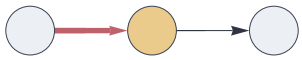
Inhibitory signals are delayed much like excitatory elements. Here, the outgoing path is tagged as a multiset edge. Therefore, it preserves the negatives:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [{"slot": 11, "tags": ["inhibit"]}]},
    {"component": "output", "receive": [{"slot": 21, "tags": ["multiset"]}]}
  ]}
```

```
> {1, 2, 3, -3, -4, -5}
{-5, -4, -3, -3, -2, -1}
```

Here the outbound, non-inhibitory pathway drops all inhibitory elements:

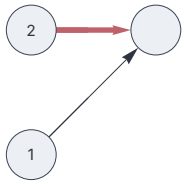
```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [{"slot": 11, "tags": ["inhibit"]}]},
    {"component": "output", "receive": [21]}
  ]}
```



```
> {1, 2, 3, -3, -4, -5}
{}
```

In this circuit, the second input inhibits the first input:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11], "label": "#"},{"component": "input", "send": [12],
  "label": "#"},{"component": "output", "receive": [[11, {"slot": 12, "tags": ["inhibit"]}]]}
  ]}
```

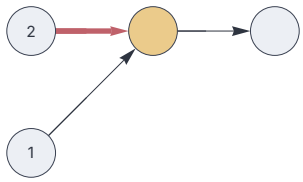


The output contains both excitatory and inhibitory elements:

```
> {1, 2, 3, 4, 5, 6}, {1, 1, 2, 2, 3, 4}
{-2, -1, 5, 6}
```

Here, the inhibitory elements are dropped in the outgoing path:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11], "label":
  "⌗"},{"component": "input", "send": [12], "label": "⌗"},{"component":
  "delay", "send": [21], "receive": [[11, {"slot": 12, "tags": ["inhibit"]}]]},
    {"component": "output", "receive": [21]}
  ]}
```



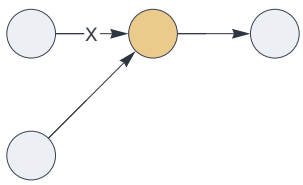

```
> {1, 2, 3, 4, 5, 6}, {1, 1, 2, 2, 3, 4}
{5, 6}
```

Data Routing and Merging

Veto signals

A path tagged with “veto” acts as a veto signal: If it contains any element, the entire bundle becomes zero:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "input", "send": [12]},
    {"component": "delay", "send": [21], "receive": [[11, {"slot": 12, "tags": ["veto"]}]]},
    {"component": "output", "receive": [21]}
  ]}
```



No veto signal:

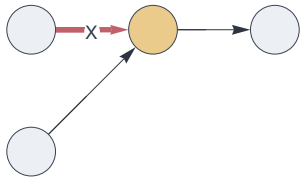
```
> {1, 2, 3, 4, 5}, {}
{1, 2, 3, 4, 5}
```

A single-element veto signal:

```
> {1, 2, 3, 4, 5}, {77}
{}
```

Also inhibitory signals can function as a veto signal:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "input", "send": [12]},
    {"component": "delay", "send": [21],
  "receive": [[11, {"slot": 12, "tags": ["inhibit", "veto"]}]]},
    {"component": "output", "receive": [21]}
  ]}
```




```
> {1, 2, 3, 4, 5}, {}
{1, 2, 3, 4, 5}
```

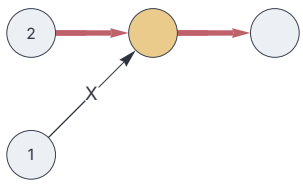
Inhibitory veto signal:

```
> {1, 2, 3, 4, 5}, {77}
{}
```

```
> {1, 2, 3, 4, 5}, {-77}
{}
```

Here, an excitatory path functions as a veto signal on an inhibitory path:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11], "label": "#"},{"component": "input",
  "send": [12], "label": "#"},{"component": "delay", "send": [21], "receive":
  [[{"slot": 11, "tags": ["veto"]}, {"slot": 12, "tags": ["inhibit"]}]]},
    {"component": "output", "receive": [{"slot": 21, "tags": ["inhibit"]}]}
  ]}
```



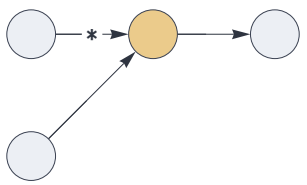
```
> {}, {-5, -4, -3, -2, -1}
{-5, -4, -3, -2, -1}
```

```
> {77}, {-5, -4, -3, -2, -1}
{}
```

Mandatory signals

If a path tagged as “mandatory” carries no signal, all merging paths are cleared.

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "input", "send": [12]},
    {"component": "delay", "send":
  [21], "receive": [[11, {"slot": 12, "tags": ["mandatory"]}]]},
    {"component": "output", "receive": [21]}
  ]}
```



Here the mandatory signal is present:

```
> {1, 2, 3, 4, 5}, {1, 2}
{1, 2, 3, 4, 5}
```

Empty mandatory signal:

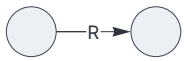
```
> {1, 2, 3, 4, 5}, {}
{}
```

Stochastic Subsampling

Stochastic subsampling is a fundamental mechanism operating on sparse binary representations. Within this software framework, subsampling mechanisms are integrated into various component types, as well as directly into the data transport mechanism (pathways).

A path tagged with “rate_limit” reduces the signal to at most it’s specified population. This mechanism acts as a rate limiter during signal transport:

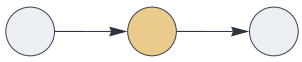
```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [{"slot": 11, "tags": ["rate_limit"]}]}
  ]}
```




```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
{1, 5, 6, 9, 10}
```

An explicit rate limit can be specified for delay components. In the following example, the absolute ratelimit 7 is obtained by multiplying the inbound population P=5 with the specified factor 1.4:

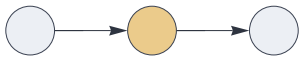
```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [11], "rate_limit": 1.4},
    {"component": "output", "receive": [21]}
  ]}
```



```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
{2, 3, 4, 5, 6, 9, 10}
```

This delay component proportionally decimates the signal by a constant factor:

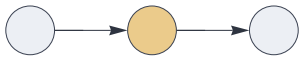
```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [11], "decimate": 0.8},
    {"component": "output", "receive": [21]}
  ]}
```



```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
{1, 3, 4, 6, 7, 8, 9, 10}
```

A delay node can apply proportional decimation and subsequent rate limiting within the same cycle:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [11], "decimate": 0.8, "rate_limit": 2.0},
    {"component": "output", "receive": [21]}
  ]}
```



```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
{1, 2, 3, 4, 5, 6, 8, 9}
```

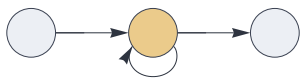
```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}
{2, 5, 6, 8, 14, 15, 18, 20}
```

Recurrence

Recurrence -- a fundamental design principle in cognitive architectures -- is characterized by cyclic connections in the circuit graph, integrating data across synchronized clock cycles through feedback loops. Any such feedback system relies on stochastic signal decimation to prevent runaway activity. This framework supports various mechanisms for managing sparsity in recurrent architectures: stochastic rate limiting located in the data transport logic (pathways) or
within delay nodes, k-winners-take-all mechanims applied in pathway integration or in temporally integrated signals, and the update rules that govern the recurrent network encapsulated within auto-associative memory components.

Here is a most basic recurrent circuit consisting of a single delay node. The two incoming signals are merged into a multiset. Rate limiting is applied within the circuit component.

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [[11, 21]], "rate_limit": 2.0},
    {"component": "output", "receive": [21]}
  ]}
```



```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

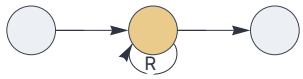
```
> {6, 7, 8, 9, 10}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

```
> {11, 12, 13, 14, 15}
{2, 3, 4, 5, 9, 10, 11, 12, 13, 15}
```

```
> {16, 17, 18, 19, 20}
{2, 4, 9, 10, 12, 13, 15, 16, 17, 18}
```

The following circuit decimates the feedback signal to the population specified as hyperparameter. At each cycle, the forward signal passes through without decimation. Note that this behavior is subtly different than the example above.

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21],
  "receive": [[11, {"slot": 21, "tags": ["rate_limit"]}]]},
    {"component": "output", "receive": [21]}
  ]}
```




```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

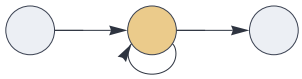
```
> {6, 7, 8, 9, 10}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

```
> {11, 12, 13, 14, 15}
{2, 3, 5, 6, 9, 11, 12, 13, 14, 15}
```

```
> {16, 17, 18, 19, 20}
{9, 11, 13, 14, 15, 16, 17, 18, 19, 20}
```

Without limiting the signal flow, the accumulated feedback would quickly outgrow the system’s sparsity envelope.

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [[11, 21]]},
    {"component": "output", "receive": [21]}
  ]}
```



```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

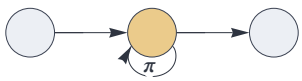
```
> {6, 7, 8, 9, 10}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

```
> {11, 12, 13, 14, 15}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15}
```

```
> {16, 17, 18, 19, 20}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}
```

Applying a permutation to the feedback signal encodes its temporal position:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive":
  [[11, {"slot": 21, "tags": ["permute"]}]], "rate_limit": 2.0},
    {"component": "output", "receive": [21]}
  ]}
```




```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

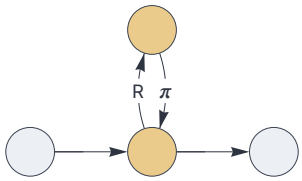
```
> {6, 7, 8, 9, 10}
{6, 7, 8, 9, 10, 413, 564, 870, 925, 981}
```

```
> {11, 12, 13, 14, 15}
{13, 14, 15, 25, 42, 198, 302, 530, 687, 846}
```

```
> {16, 17, 18, 19, 20}
{17, 20, 95, 133, 253, 348, 638, 798, 814, 941}
```

Here is a recurrent circuit involving two delays:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [[11, {"slot": 22, "tags": ["permute"]}]]},
    {"component": "delay", "send": [22], "receive": [{"slot": 21, "tags": ["rate_limit"]}]},
    {"component": "output", "receive": [21]}
  ]}
```




```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

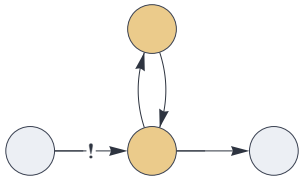
```
> {6, 7, 8, 9, 10}
{6, 7, 8, 9, 10}
```

```
> {11, 12, 13, 14, 15}
{11, 12, 13, 14, 15, 180, 412, 594, 828, 883}
```

```
> {16, 17, 18, 19, 20}
{16, 17, 18, 19, 20, 104, 169, 349, 720, 975}
```

The following shows a reverberatory circuit that bounces signals between two delay nodes, toggling two states in the absence of new input. As the incoming pathway takes priority over the feedback, no rate decimation is needed.

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "delay", "send": [21], "receive": [[{"slot": 11, "tags": ["priority"]}, 22]]},
    {"component": "delay", "send": [22], "receive": [21]},
    {"component": "output", "receive": [21]}
  ]}
```



```
> {1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

```
> {6, 7, 8, 9, 10}
{6, 7, 8, 9, 10}
```

```
> {}
{1, 2, 3, 4, 5}
```

```
> {}
{6, 7, 8, 9, 10}
```

```
> {}
{1, 2, 3, 4, 5}
```

Noise Generation

The built-in “noise” component generates a pseudo-random set at every execution cycle. The number of elements is taken to be the population specified for the outbound path:

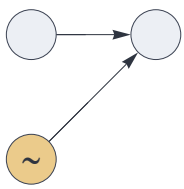
```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "noise", "send": [11]},
    {"component": "output", "receive": [11]}
  ]}
```

This generates a set with 5 random elements:

```
>
{7, 46, 733, 801, 986}
```

This circuit adds noise to the input. Note that the set notation [11,12] indicates pathway merging, or bundling:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "noise", "send": [11]},
    {"component": "input", "send": [12]},
    {"component": "output", "receive": [[11, 12]]}
  ]}
```



The input and the random signal have a small overlap probability.

```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 146, 159, 163, 194, 650}
```

This generates a random inhibitory signal:

```
{ "hyperparameters": {"default": [1000, 20]},
  "dataflow": [
    {"component": "noise", "send": [11]},
    {"component": "output", "receive": [{"slot": 11, "tags": ["inhibit"]}]}
  ]}
```

```
>
{-961, -925, -862, -858, -816, -777, -754, -730,
  -650, -648, -600, -536, -389, -361, -261, -237, -180, -120, -107, -45}
```

A pathway tagged as “noise” generates random noise if it receives an empty input, and clears all data if the input is non-empty.

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [{"slot": 11, "tags": ["noise"]}]}
  ]}
```

```
> {}
{201, 305, 322, 908, 923}
```

```
> {1, 2, 3, 4, 5}
{}
```

K-Winners-Take-All

K-winner-take-all (kWTA) is a competitive neural network mechanism where only the top k most active neurons are allowed to fire, while all others are suppressed to zero. Translated to this computational framework, kWTA converts a multiset to a set. This ubiquitous algorithm drives the topological associative memory algorithm, is used in SDR encoding, and path merging. In addition, it is implemented as a circuit component which applies to kWTA to signals within a sliding time window.

This basic circuit applies kWTA to a multiset pathway:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "output", "receive": [{"slot": 11, "tags": ["kwta_absolute", "multiset"]}]}
  ]}
```


```
> {1, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 7, 8, 9, 10}
{1, 2, 3, 4, 5}
```

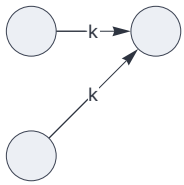
Elements must occur at least twice in order to survive.

```
> {1, 2, 3, 4, 5}
{}
```

Path merging with kWTA

Merging paths where at least one path is tagged with “kwta_absolute” (displayed as “k”) applies the kWTA algorithm, using the path’s population parameter as the ranking parameter.

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "input", "send": [12]},
    {"component": "output", "receive": [[{"slot": 11,
  "tags": ["kwta_absolute"]}, {"slot": 12, "tags": ["kwta_absolute"]}]]}
  ]}
```

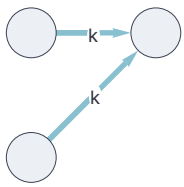


This isolates k=5 elements that occur at least twice.

```
> {1, 2, 3, 4, 5, 6, 7}, {3, 4, 5, 6, 7, 8, 10}
{3, 4, 5, 6, 7}
```

Here two multiset paths are merged into a proper set:

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "input", "send": [12]},
    {"component": "output", "receive": [[{"slot": 11, "tags": ["multiset",
  "kwta_absolute"]}, {"slot": 12, "tags": ["multiset", "kwta_absolute"]}]]}
  ]}
```



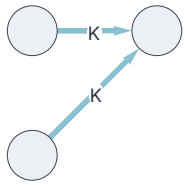
Both inputs are multisets:

```
> {1, 1, 2, 3, 5, 6, 7}, {2, 3, 4, 4, 5, 8, 9}
{1, 2, 3, 4, 5}
```

The kwta algorithm is applied equally to excitatory and inhibitory signals:

```
> {1, 1, 2, 3, 5, 6, 7, -11}, {2, 3, 4, 4, 5, 8, 9, -11}
{-11, 1, 2, 3, 4, 5}
```

```
{ "hyperparameters": {"default": [1000, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "input", "send": [12]},
    {"component": "output", "receive": [[{"slot": 11, "tags": ["multiset",
  "kwta_excitatory"]}, {"slot": 12, "tags": ["multiset", "kwta_excitatory"]}]]}
  ]}
```




```
> {1, 1, 2, 3, 5, 6, 7}, {2, 3, 4, 4, 5, 8, 9}
{1, 2, 3, 4, 5}
```

```
> {1, 1, 2, 3, 5, 6, 7, -11}, {2, 3, 4, 4, 5, 8, 9, -11}
{1, 2, 3, 4, 5}
```

Auto-Associative Memory

Auto-associative memory components can be configured with various update rules that govern how the memory's input/output layer X is updated with the pattern Y retrieved from memory. This table outlines the most common methods:

▼	replacement	X ← Y
▲	residual	X ← X \ Y
▽	complement	X ← Y \ X
△	difference	X ← X △ Y
∩	coincidence	X ← X ∩ Y
∪	augmentation	X ← X ∪ Y

Replacement update

By default, The internal recurrent architecture of this memory component replaces its state with the output of the memory query ("replacement"). This setup functions as item memory, completing inputs and removing noise. It can be used with sparse holographic representations (SHRs) of symbolic tokens, or with sparse distributed representations (SDRs) representing perceptual data.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "auto", "plugin": "replacement", "send": [21], "receive": [11]},
    {"component": "output", "receive": [21]}
  ]}
```

Learn a pattern:

```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

Retrieve from partial input:

```
> {1, 2, 3, 4, 5, 6, 7}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

Recall from noisy input:

```
> {1, 2, 3, 4, 5, 6, 7, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

This input resolves to a known pattern within the default pattern matching treshold. The item memory does not learn it:

```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 17}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

Learn a new pattern:

```
> {11, 12, 13, 14, 15, 16, 17, 18, 19, 20}
{11, 12, 13, 14, 15, 16, 17, 18, 19, 20}
```

Memory retrieval finds the unique best partial match:

```
> {1, 2, 3, 4, 5, 6, 7, 8, 11, 12, 13, 14, 15, 16, 17, 18, 19}
{11, 12, 13, 14, 15, 16, 17, 18, 19, 20}
```

This results in a tie: no unique best match exists.

```
> {1, 2, 3, 4, 5, 6, 7, 8, 11, 12, 13, 14, 15, 16, 17, 18}
{}
```

Memory retrieval requires a minimum number of matching elements. In this configuration, the pattern matching threshold is automatically set to T = 7.

```
> {1, 2, 3, 4, 5, 6}
{}
```

An item memory automatically learns a new input if memory retrieval within the matching threshold fails:

```
> {1, 2, 3, 4, 5, 6, 107, 108, 109, 110}
{1, 2, 3, 4, 5, 6, 107, 108, 109, 110}
```

Below is an item memory for sparse distributed representations (SDRs), using an explicit threshold setting. As no update rule is specified in the auto-associative memory component, the default “replacement” is assumed.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "auto", "send": [21], "receive": [11], "threshold": 1.0},
    {"component": "output", "receive": [21]}
  ]}
```

In the this example, the effective pattern matching threshold is T=10 (1.0 times the specified population).

Learn a pattern:

```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

Memory retrieval below the matching threshold fails:

```
> {1, 2, 3, 4, 5, 6, 7, 8, 9}
{}
```

Learn a pattern that differs from the previous one by just one element:

```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 50}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 50}
```

This retrieves the exact match from noisy input:

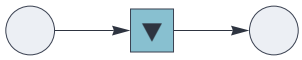
```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 50, 51, 52, 53, 54, 55, 56}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 50}
```

Input below the matching threshold:

```
> {2, 3, 4, 5, 6, 7, 8, 9, 50, 51, 52, 53, 54, 55, 56}
{}
```

By default, an auto-associative item store learns a new pattern only if its length matches the population specified in the outgoing path. Here we specify a custom range:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "auto", "send": [21], "receive": [11], "learn": [1.0, 1.5]},
    {"component": "output", "receive": [21]}
  ]}
```



```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11}
```

Retrieve this item from partial input:

```
> {1, 2, 3, 4, 5, 6, 7}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11}
```

Set up an item memory for symbolic tokens, encoded as SHRs:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [11]},
    {"component": "auto", "send": [21], "receive": [11]},
    {"component": "output", "plugin": "codec", "receive": [21]}
  ]}
```


```
> pineapple
pineapple
```

Residual update

Auto-associative memory using the “residual” update rule returns the residual input elements that remain after dropping the elements retrieved from memory.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "auto", "plugin": "residual", "send": [21], "receive": [11]},
    {"component": "output", "receive": [21]}
  ]}
```

Learn a pattern:

```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
{}
```

Memory retrieval:

```
> {1, 2, 3, 4, 5, 6, 7, 100, 101, 102}
{100, 101, 102}
```

Complement update

The “complement” update rule removes the input from the

retrieved pattern.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "auto", "plugin": "complement", "send": [21], "receive": [11]},
    {"component": "output", "receive": [21]}
  ]}
```

Learn a pattern. Note that this returns an empty result.

```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
{}
```

Retrieval:

```
> {1, 2, 3, 4, 5, 6, 7, 100, 101, 102}
{8, 9, 10}
```

Difference update

Auto-associative memory using the “difference” update rule replaces the state with the symmetric difference between the query pattern and the recalled pattern.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "auto", "plugin": "difference", "send": [21], "receive": [11]},
    {"component": "output", "receive": [21]}
  ]}
```

Learn a pattern. This returns an empty result.

```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
{}
```

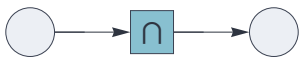
Retrieval:

```
> {1, 2, 3, 4, 5, 6, 7, 100, 101, 102}
{8, 9, 10, 100, 101, 102}
```

Coincidence update

The “coincidence” update rule replaces the state with the symmetric difference between the query pattern and the recalled pattern.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "auto", "plugin": "coincidence", "send": [21], "receive": [11]},
    {"component": "output", "receive": [21]}
  ]}
```



Learn a pattern:

```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

Retrieve:

```
> {1, 2, 3, 4, 5, 6, 7, 100, 101, 102}
{1, 2, 3, 4, 5, 6, 7}
```

Augmentation update

The “augmentation” update rule returns the union of the query and the retrieved pattern.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "auto", "plugin": "augmentation", "send": [21], "receive": [11]},
    {"component": "output", "receive": [21]}
  ]}
```

Learn a pattern:

```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

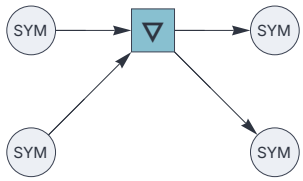
Retrieve:

```
> {1, 2, 3, 4, 5, 6, 7, 100, 101, 102}
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 100, 101, 102}
```

Bidirectional Memory

Set up a bidirectional memory as a bi-partite auto-associative memory. This uses the “complement” update rule, which removes the input pattern from the recalled output.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [11]},
    {"component": "input", "plugin": "codec", "send": [12]},
    {"component": "auto", "plugin": "complement", "send": [21, 22], "receive": [11, 12]},
    {"component": "output", "plugin": "codec", "receive": [21]},
    {"component": "output", "plugin": "codec", "receive": [22]}
  ]}
```



Learn a pair of symbols:

```
> blue, sky
Null, Null
```

Query with the first token:

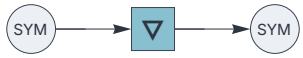
```
> blue, Null
Null, sky
```

Reverse query:

```
> Null, sky
blue, Null
```

This setup realizes a bidirectional associative memory, representing token pairs as bundles.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [11]},
    {"component": "auto", "plugin": "complement",
  "send": [21], "receive": [11], "threshold": 1.0, "learn": [1.8, 2.0]},
    {"component": "output", "plugin": "codec", "receive": [21]}
  ]}
```



Learn an unordered token pair. Note that the symbol encoder maps over the list of inputs and bundles the resulting sets.

```
> {big, small}
Null
```

Retrieval with one of the tokens:

```
> big
small
```

Retrieval with the other token:

```
> small
big
```

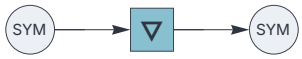
The memory does not learn this novel input because a partial match with known information is found:

```
> {big, apple}
small
```

Hyper-Associative Memory

This sets up an auto-associative memory using the “complement” update rule. The system learns new information if it lies in the specified population range:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [11]},
    {"component": "auto", "plugin":
  "complement", "send": [21], "receive": [11], "learn": [1.0, 1.0e1]},
    {"component": "output", "plugin": "codec", "receive": [21]}
  ]}
```



Store a bundle of five tokens:

```
> {A, B, C, D, E}
Null
```

Querying with one token:

```
> {A}
{B, C, D, E}
```

Querying with any two tokens:

```
> {A, E}
{B, C, D}
```

A full match gives an empty result:

```
> {A, B, C, D, E}
Null
```

Partial matching:

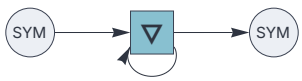
```
> {A, B, C, D, E, X}
Null
```

No new information was learned in the previous step.

```
> {A, B, C}
{D, E}
```

Generative behavior emerges if we feed the output of an auto-associative memory back into its input. This example uses the "complement" update rule:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [11]},
    {"component": "auto", "plugin": "complement",
  "send": [21], "receive": [[11, 21]], "learn": [1.0, 1.0e1]},
    {"component": "output", "plugin": "codec", "receive": [21]}
  ]}
```



Store a bundle of five tokens:

```
> {A, B, C, D, E}
Null
```

Querying with two tokens:

```
> {A, B}
{C, D, E}
```

Empty input triggers a generative loop based on the feedback:

```
> {}
{A, B}
```

```
> {}
{C, D, E}
```

Merge external input with the feedback:

```
> {B}
A
```

A full match gives an empty result:

```
> {A, B, C, D, E}
Null
```

Partial match:

```
> {A, B, C, D, E, X}
Null
```

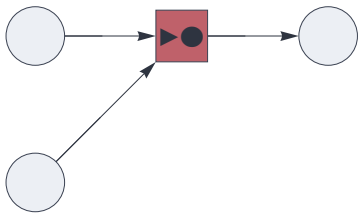
No new information was learned in the previous step.

```
> {A, B, C}
{D, E}
```

Hetero-associative memory

Here is a basic hetero-associative memory circuit, taking two inputs and generating one output:

```
{ "hyperparameters": {"11": [1000, 8], "default": [500, 5]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "input", "send": [12]},
    {"component": "associator", "send": [101], "receive": [12, 11]},
    {"component": "output", "receive": [101]}
  ]}
```



Store an association:

```
> {1, 2, 3, 4, 5, 6, 7, 8}, {51, 52, 53, 54, 55}
{}
```

Memory retrieval:

```
> {1, 2, 3, 4, 5, 6, 7, 8}, {}
{51, 52, 53, 54, 55}
```

Store a second association:

```
> {11, 12, 13, 14, 15, 16, 17, 18}, {61, 62, 63, 64, 65}
{}
```

Retrieval with the union of previous two patterns results in a pattern matching tie:

```
> {1, 2, 3, 4, 5, 6, 7, 8, 11, 12, 13, 14, 15, 16, 17, 18}, {}
{}
```

Break the tie by dropping one element:

```
> {2, 3, 4, 5, 6, 7, 8, 11, 12, 13, 14, 15, 16, 17, 18}, {}
{61, 62, 63, 64, 65}
```

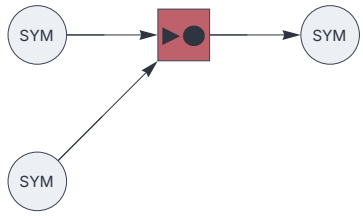
Supervised Learning with Heteroassociations

The “associator” node implements a basic hetero-associative memory for supervised learning applications. It stores associations of the form A→B, where A and B are typically sets from different spaces with different hyperparameters.

Heteroassociations may involve perceptual (SDR) or symbolic (SHR) data.

This basic example demonstrates heteroassociations between symbolic representations. Note that the training signal (B) is the first argument of the associator node:

```
{ "hyperparameters": {"11": [1000, 10], "default": [1500, 15]},
  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [11]},
    {"component": "input", "plugin": "codec", "send": [21]},
    {"component": "associator", "send": [101], "receive": [21, 11]},
    {"component": "output", "plugin": "codec", "receive": [101]}
  ]}
```



Learn A → B:

```
> A, B
Null
```

A query with A retrieves B:

```
> A, Null
B
```

The reverse query fails, as the memory is uni-directional.

```
> Null, B
Null
```

Retrieve with noisy input. Here the encoder bundles the representations for A and X.

```
> {A, X}, Null
B
```

Storing A → C extends the previously stored association for A:

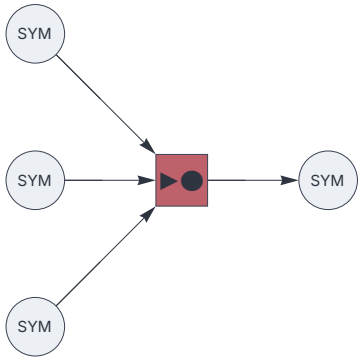
```
> A, C
Null
```

Querying with A now returns a bundle:

```
> A, Null
{B, C}
```

The left-hand side of a heteroassociation A → B can be partitioned into blocks. Here we set up a dataflow with a bipartite input. The pattern matching threshold T is configured to be greater than the population of a single partition:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [11]},
    {"component": "input", "plugin": "codec", "send": [12]},
    {"component": "input", "plugin": "codec", "send": [21]},
    {"component": "associator", "send": [101], "receive": [21, 11, 12], "threshold": 0.6},
    {"component": "output", "plugin": "codec", "receive": [101]}
  ]}
```



Learn “A” + “context1” → B:

```
> A, context1, B
Null
```

Memory retrieval:

```
> A, context1, Null
B
```

Retrieval with an unknown pattern “context2”:

```
> A, context2, Null
Null
```

Learn an association involving “context2”:

```
> A, context2, C
Null
```

Memory retrieval:

```
> A, context2, Null
C
```

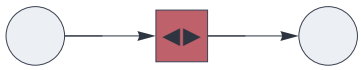
This query fails because a pattern matching tie occurs:

```
> A, {context1, context2}, Null
Null
```

Heteroencoding

A heteroencoder is a hetero-associative memory component that clusters sparse distributed representations (SDRs) based on their similarity. It creates a random sparse holographic representation (SHR) for each cluster identified in the input data.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "heteroencoder", "send": [101], "receive": [11]},
    {"component": "output", "receive": [101]}
  ]}
```



This generates a random SHR, representing a new data cluster:

```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
{94, 222, 342, 377, 411, 413, 547, 687, 821, 955}
```

Similar input is mapped to the same cluster:

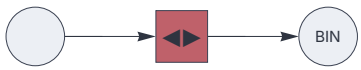
```
> {2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12}
{94, 222, 342, 377, 411, 413, 547, 687, 821, 955}
```

Dissimilar input is mapped to a new cluster:

```
> {21, 22, 23, 24, 25, 26, 27, 28, 29, 30}
{63, 264, 332, 344, 442, 466, 515, 528, 759, 825}
```

Use the “binning” decoder to automatically map clusters to consecutive numbers:

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "send": [11]},
    {"component": "heteroencoder", "send": [101], "receive": [11]},
    {"component": "output", "plugin": "binning", "receive": [101]}
  ]}
```




```
> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
1
```

Similar input:

```
> {2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12}
1
```

Dissimilar input:

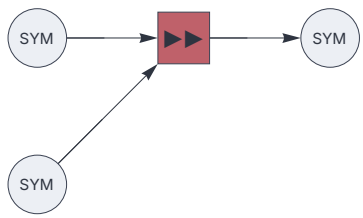
```
> {21, 22, 23, 24, 25, 26, 27, 28, 29, 30}
2
```

Predictive Associations

The “predictor” node is a hetero-associative memory component that predicts its next input based on the current input and a context. The context is typically a readout from a recurrent reservoir network.

Note that the token to be learned is given as the first argument of the node’s “receive” sequence (slot 11 in the example below). The remaining arguments represent the context, which may be partitioned into multiple blocks.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [11]},
    {"component": "input", "plugin": "codec", "send": [21]},
    {"component": "predictor", "send": [101], "receive": [11, 21]},
    {"component": "output", "plugin": "codec", "receive": [101]}
  ]}
```



Start a data stream:

```
> A, context1
Null
```

Learn that “B” follows “context1”:

```
> B, context2
Null
```

Learn that “A” follows “context2”:

```
> A, context1
B
```

Predict “A”:

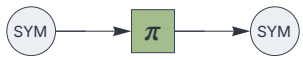
```
> Null, context2
A
```

Temporal-associative memory

Temporal-associative memory is a hybrid between auto-associative and hetero-associative architectures. It operates on a shared input/output layer, and associates its state to the next input, acting as a next-token predictor. The temporal integration mechanics is shared with the “delay” component.

By default, the “temporal” component accumulates its state through repeated permutation, enabling to learn long token sequences on the fly.

```
{ "hyperparameters": {"default": [1000, 10]},
  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [11]},
    {"component": "temporal", "send": [21], "receive": [11], "capacity": 3.0, "decimate": 0.8},
    {"component": "output", "plugin": "codec", "receive": [21]}
  ]}
```




```
> A
Null
```

```
> B
Null
```

```
> A
B
```

```
> A
A
```

```
> B
A
```